

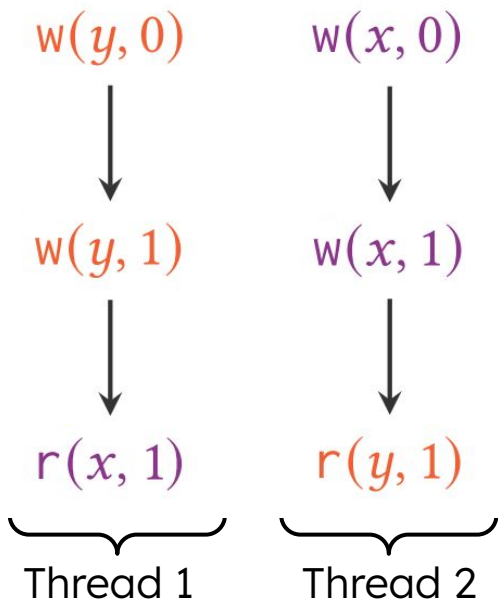


An MSO Framework for Weak-Memory Verification and Robustness

GIOVANNA KOBUS CONRADO, ANDREAS PAVLOGIANNIS

Aarhus University

How do we model shared memory in a concurrent program?

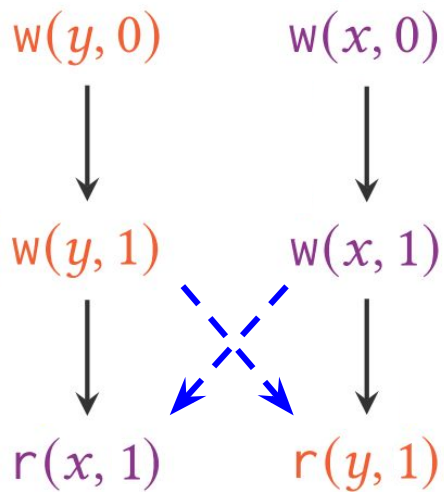


Events **E**: $w(x, 0)$ $w(y, 1)$ $w(x, 1)$
 $w(y, 0)$ $r(x, 0)$ $r(y, 0)$

Sequenced-before relation sb:



How do we model shared memory in a concurrent program?



Events **E**: $w(x, 0)$ $w(y, 1)$ $w(x, 1)$
 $w(y, 0)$ $r(x, 0)$ $r(y, 0)$

Sequenced-before relation **sb**:

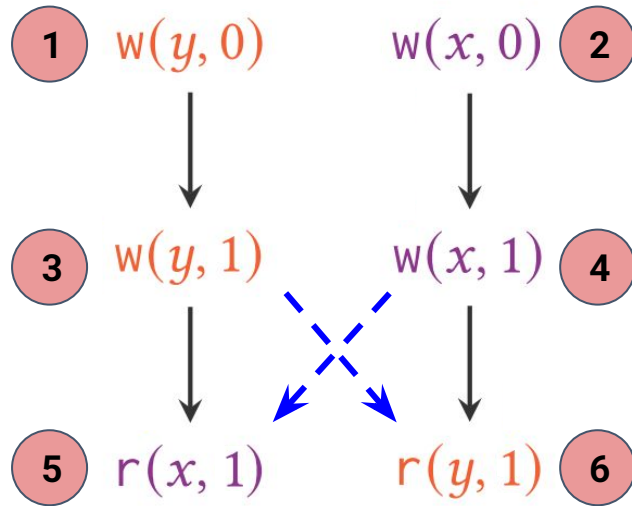


Reads-from relation **rf**:



We call this an **rf-graph**.

How do we model shared memory in a concurrent program?



Events **E**: $w(x, 0)$ $w(y, 1)$ $w(x, 1)$
 $w(y, 0)$ $r(x, 0)$ $r(y, 0)$

Sequenced-before relation **sb**:

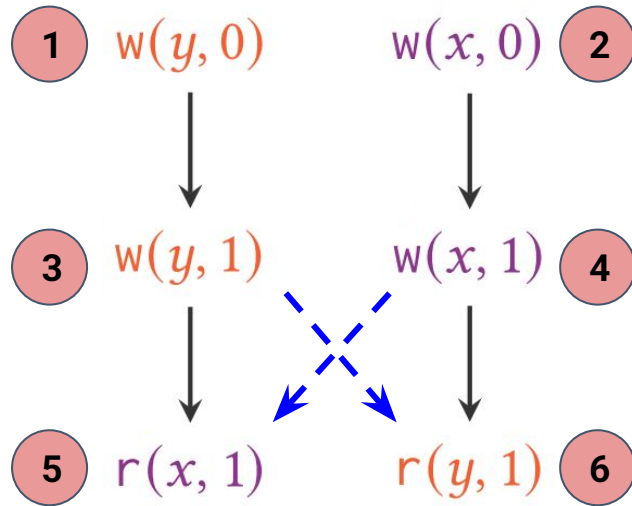


Reads-from relation **rf**:



This execution can be modelled by executing events one at a time following **sb** order.

How do we model shared memory in a concurrent program?



Events **E**: $w(x, 0)$ $w(y, 1)$ $w(x, 1)$
 $w(y, 0)$ $r(x, 0)$ $r(y, 0)$

Sequenced-before relation **sb**:



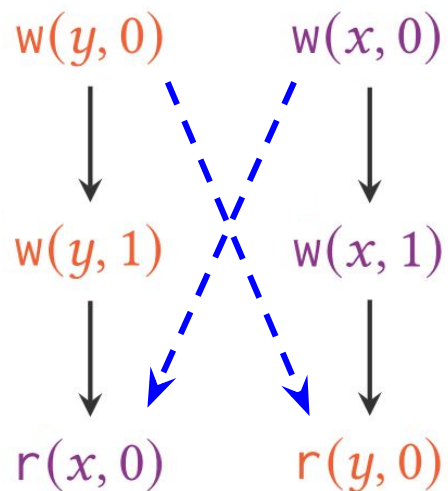
Reads-from relation **rf**:



This execution can be modelled by executing events one at a time following **sb** order.

We call this set of behaviours **sequential consistency (SC)**.

Weak memory is faster, but allows more behaviors.



Events **E**: $w(x, 0)$ $w(y, 1)$ $w(x, 1)$
 $w(y, 0)$ $r(x, 0)$ $r(y, 0)$

Sequenced-before relation sb :



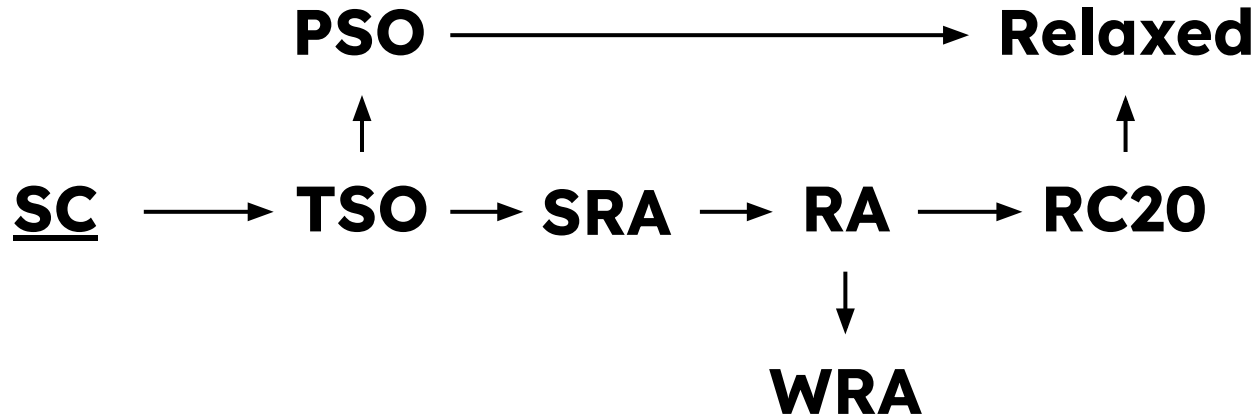
Reads-from relation rf :



This behavior is allowed, even if there is no nice order.

Weak memory models are formal specifications of these more permissive sets of allowed behaviors.

These specifications depend on language, architecture, and options chosen by the developer: **there are lots of models**



How do we deal with all these models?

A **unified metatheory** that handles problems over many memory models **uniformly**.

In this paper, we study two problems:

- **Reachability**: can a program reach a certain state when run under weak memory?
- **rf-robustness**: does a program produce the same **rf**-graphs under strong and weak memory?

MSO + treewidth as a unifying framework

A recipe for decidability

Treewidth measures how tree-like a graph is.

Monadic Second-Order Logic (MSO) is formal language that can be used to describe sets of graphs.

Courcelle's theorem:

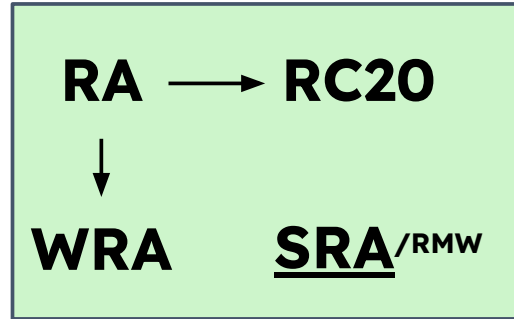
MSO satisfiability under **bounded treewidth** graphs is **decidable**

We want to use Courcelle to get **uniform decidability** results.

In this paper, we tackle the following three questions:

- Which memory models have bounded-treewidth **rf**-graphs?
- Which memory have reachability definable in MSO?
- **What are the model-checking implications?**

We demonstrate this framework by devising an algorithm that can solve reachability **or** **rf**-robustness, for these models:



Even if the problems are individually undecidable.

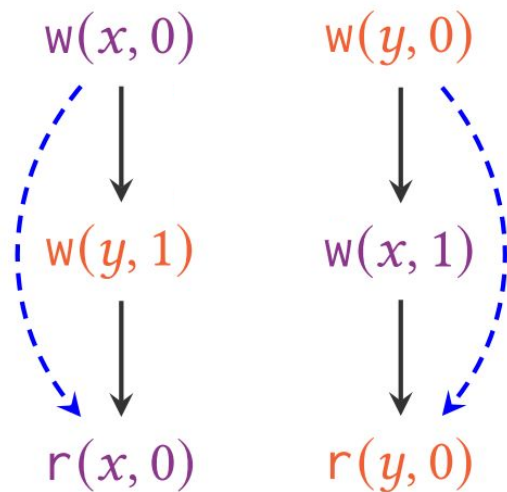
This presentation

1. **rf**-graphs and memory model consistency
2. Which memory models have bounded-treewidth **rf**-graphs?
3. Which memory have reachability definable in MSO?
4. What are the model-checking implications?
5. Takeaways

**rf-graphs and
memory model
consistency**

How do we check if an **rf**-graph is **consistent** under some memory model?

This is an **rf-graph** $\langle E, sb, rf \rangle$



Events **E**: $w(x, 0)$ $w(y, 1)$ $w(x, 1)$
 $w(y, 0)$ $r(x, 0)$ $r(y, 0)$

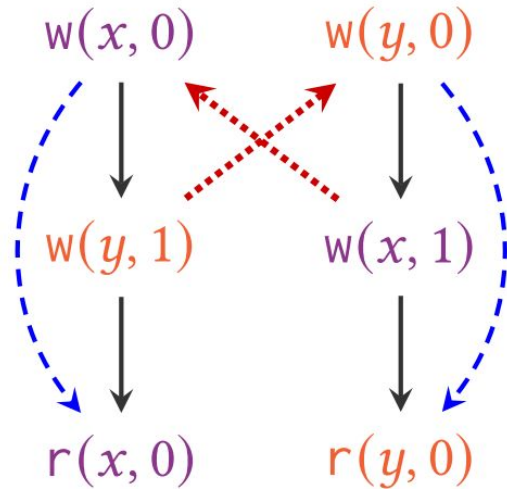
Sequenced-before relation sb :



Reads-from relation rf :



We define an additional relation **mo**



Events **E**: $w(x, 0)$ $w(y, 1)$ $w(x, 1)$
 $w(y, 0)$ $r(x, 0)$ $r(y, 0)$

Sequenced-before relation sb:



Reads-from relation rf:



Modification order **mo**:

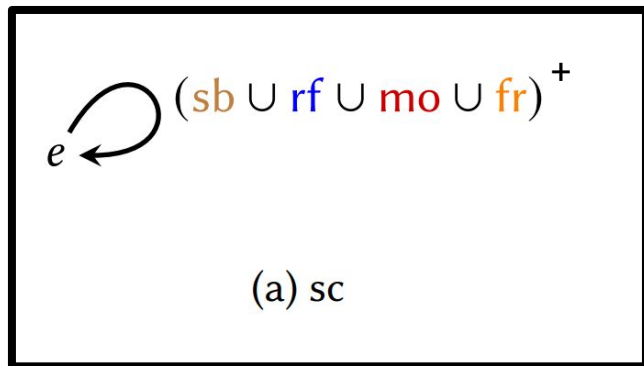


In **axiomatic semantics**, we define memory models in terms of forbidden patterns in the **rf**-graph + **mo**.

An **rf**-graph G is consistent under MM if **there exists some** **mo** such that G follows the axioms of MM

For Sequential Consistency we have the following axiom:

$$\text{fr} = \text{rf}^{-1}; \text{mo}$$



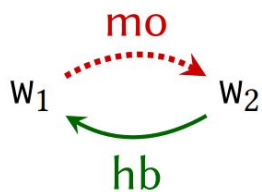
An **rf**-graph G is SC-consistent if there exists an **mo** such that G does not contain the forbidden pattern.

For Release/Acquire we have:

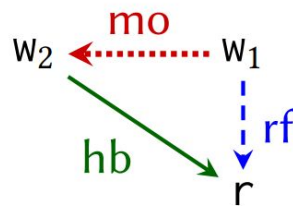
$$\text{hb} = (\text{sb} \cup \text{rf})^+$$



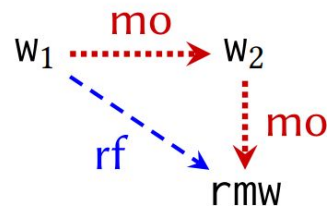
(a) irr-hb



(b) write-coherence



(c) read-coherence



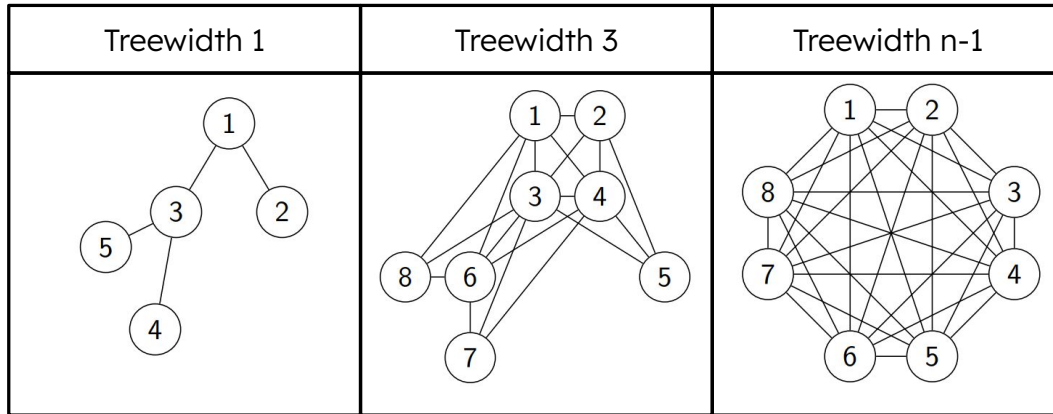
(d) atomicity

An **rf**-graph G is RA-consistent if there exists an **mo** such that G does not contain **any** of the forbidden patterns.

Which memory
models have
bounded-treewidth
rf-graphs?

What is treewidth?

Treewidth is a number measuring how tree-like a graph is.



Many problems are easier for low treewidth graphs.

Do **rf**-graphs have low treewidth?

SC-consistent **rf**-graphs

We prove that, for a program P:

The treewidth of any SC-consistent **rf-graph is bounded by $|\text{Threads}| + |\text{Locations}|$**

What about under weak memory?

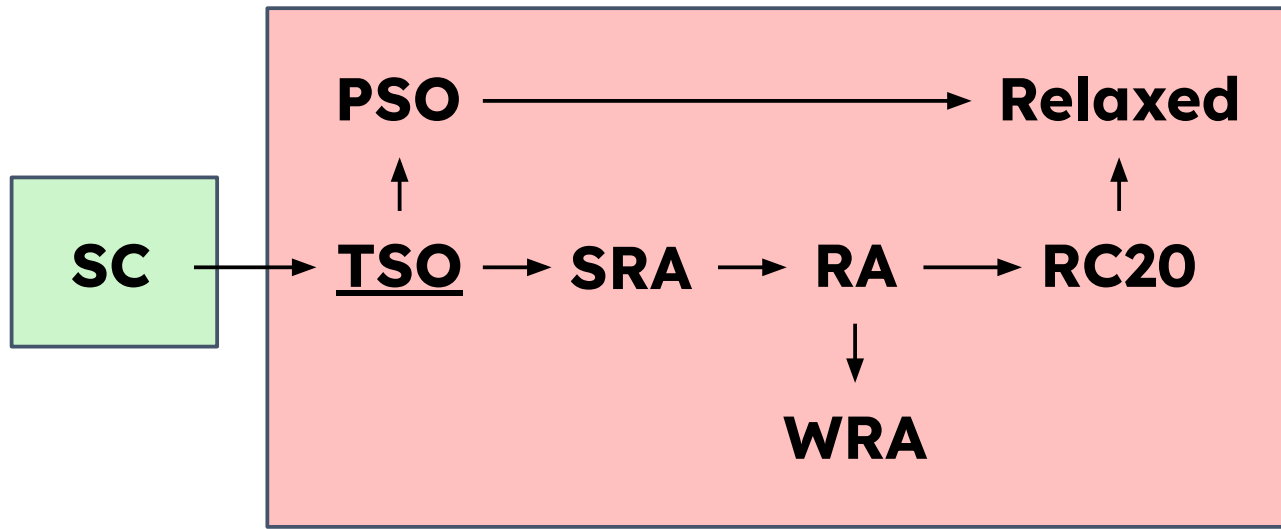
Treewidth and weak-memory

The second-strongest model we consider is **TSO**, and it already admits **rf**-graphs **G** of **unbounded treewidth**.

→ For any **k**, we show that there is **TSO**-consistent **rf**-graph with **2 threads**, **2 locations**, and **treewidth k**.

Treewidth and weak-memory

Since the treewidth of **rf**-graphs under **TSO** is unbounded, so is the treewidth of **rf**-graphs under any weaker model.



We can still use treewidth to **underapproximate** program behaviour.

Reachability: can a program reach a certain state when run under weak memory?



Bounded treewidth reachability: can a program reach a certain state through a bounded-treewidth **rf**-graph when run under weak memory?

Bounded treewidth reachability is decidable if reachability can be expressed in MSO.

Which memory have
reachability
definable in MSO?

What is MSO?

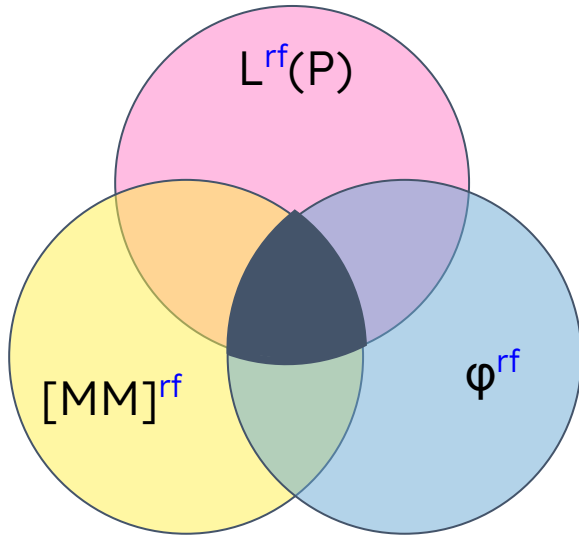
MSO formulas over **rf**-graphs are given by the following grammar, and interpreted in the natural way.

$$\varphi ::= \text{true} \mid e_1 = e_2 \mid \text{lab}(e) = \alpha \mid \text{sb}(e_1, e_2) \mid \text{rf}(e_1, e_2) \mid e \in X \mid \varphi \vee \varphi \mid \neg \varphi \mid \exists e. \varphi \mid \exists X. \varphi$$

If we can express reachability in MSO, by Courcelle, we can solve reachability for bounded-treewidth **rf**-graphs.

Reachability in MSO

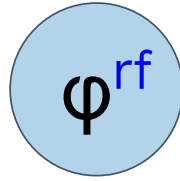
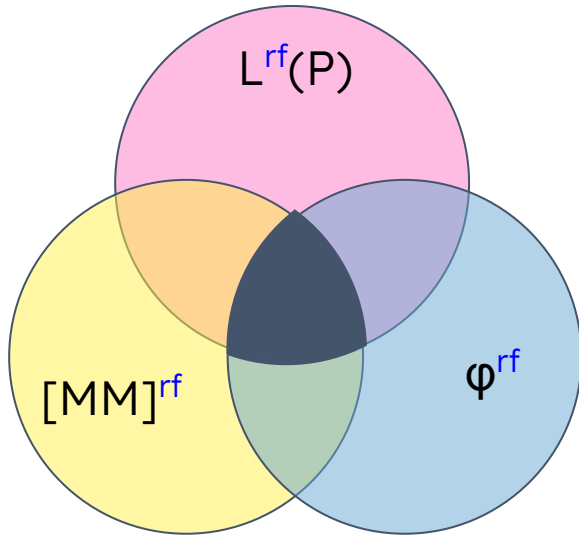
Reachability is the intersection of three sets



Since MSO supports conjunction, it is sufficient to find a formula that expresses each set

Reachability in MSO

Reachability is the intersection of three sets

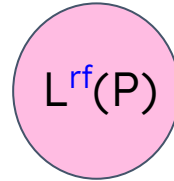
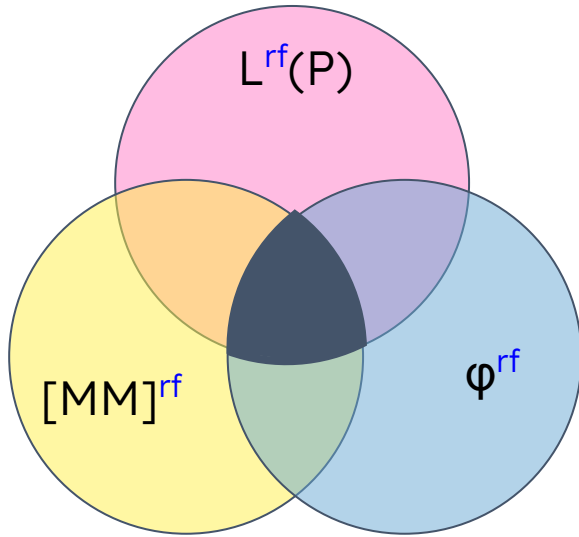


An MSO formula that expresses that **G** reaches some state

Trivial, modelling

Reachability in MSO

Reachability is the intersection of three sets

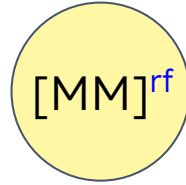
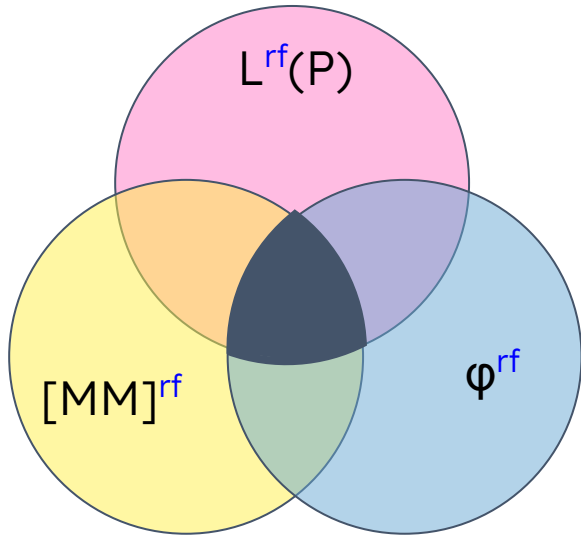


An MSO formula that expresses that **G** is an execution of **P**

Standard

Reachability in MSO

Reachability is the intersection of three sets



An MSO formula that expresses that **G** is MM-consistent

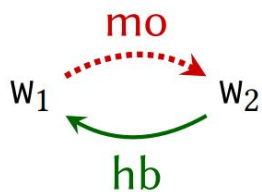
Which memory models are MSO-definable?

For Release/Acquire we have:

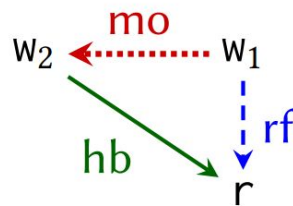
$$\mathbf{hb} = (\mathbf{sb} \cup \mathbf{rf})^+$$



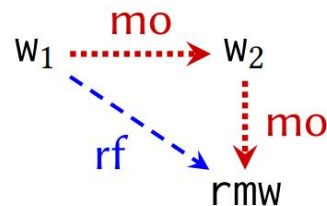
(a) irr-hb



(b) write-coherence



(c) read-coherence

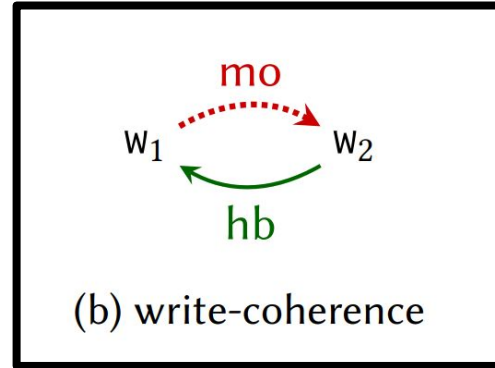
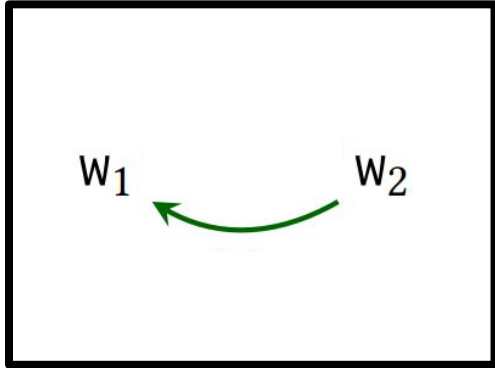


(d) atomicity

However, this would require us to existentially quantify over **mo**, which MSO does not support.

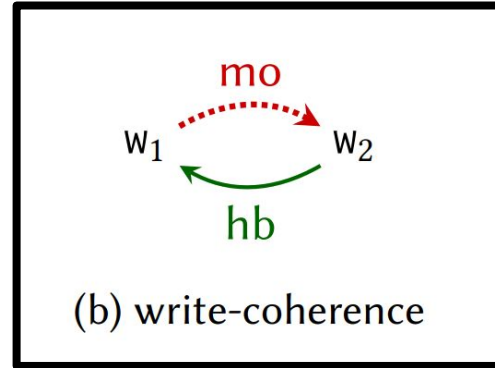
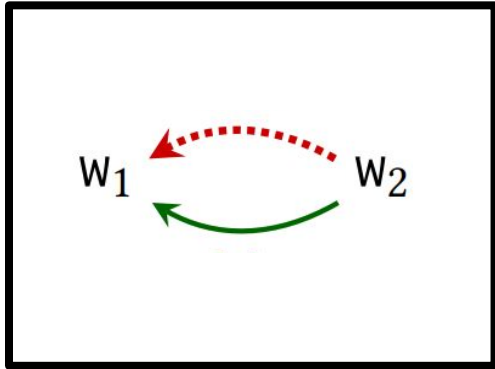
Can we infer **mo**?

Consider the following **rf**-graph and the axiom of write-coherence.



Can we infer **mo**?

Consider the following **rf**-graph and the axiom of write-coherence.



An **mo** edge between these events must go this way!

Saturation

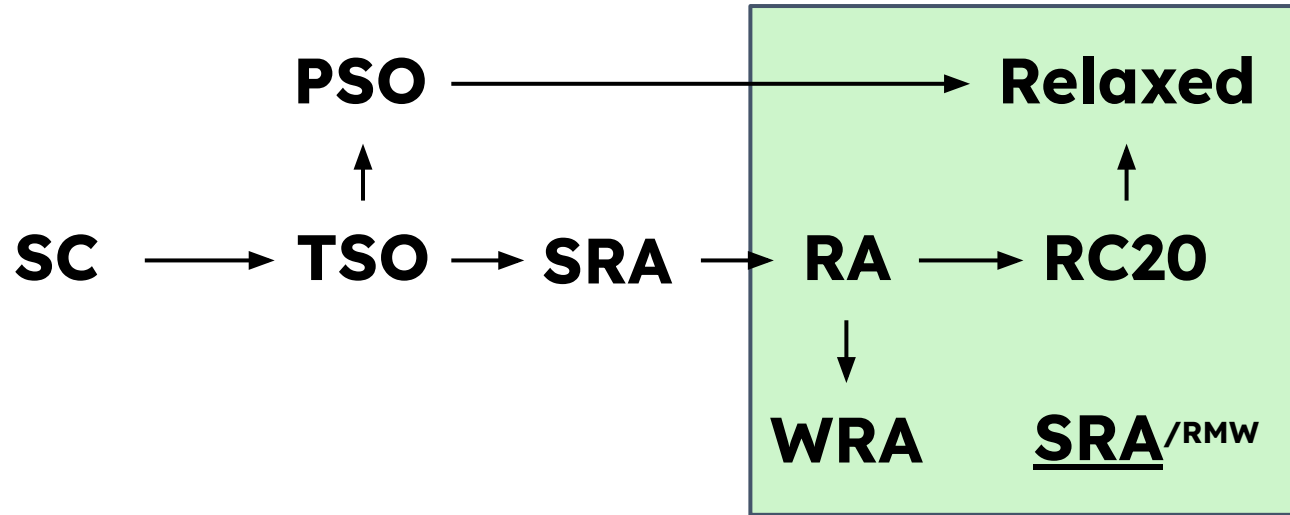
Saturation is a popular procedure used to infer **mo**-edges that is sufficient for verifying consistency.

→ Can we write this procedure in MSO?

Depends on the memory model.

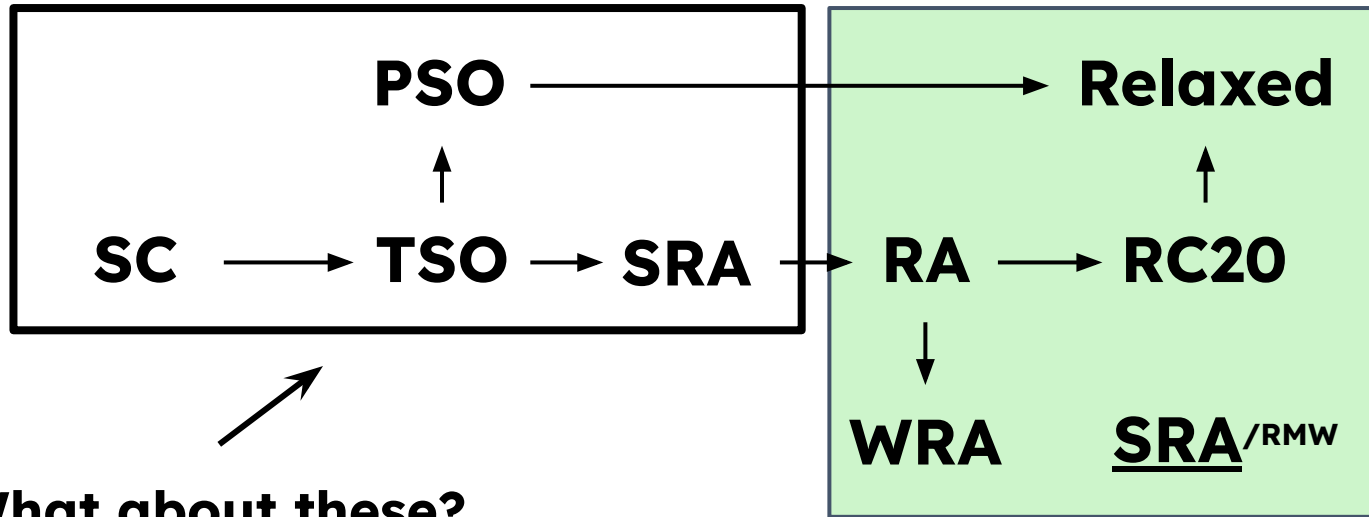
MSO-definable MMs

We define MM-consistency in MSO for the following models



MSO-definable MMs

We define MM-consistency in MSO for the following models

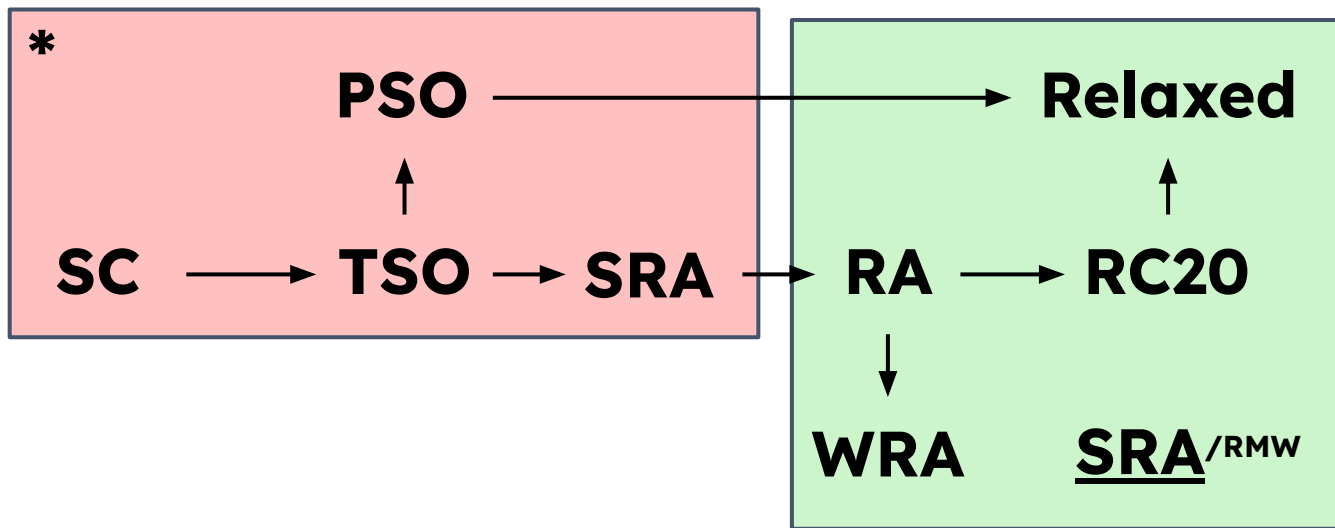


non-MSO-definable MMs

Courcelle's theorem: MSO testing on bounded treewidth graphs can be done in linear time

- OV is a problem that requires quadratic time under SETH
- We construct a bounded-treewidth **rf**-graph that is consistent in MM iff some OV instance is positive
- Therefore, MM being MSO definable would break SETH

MSO and weak memory

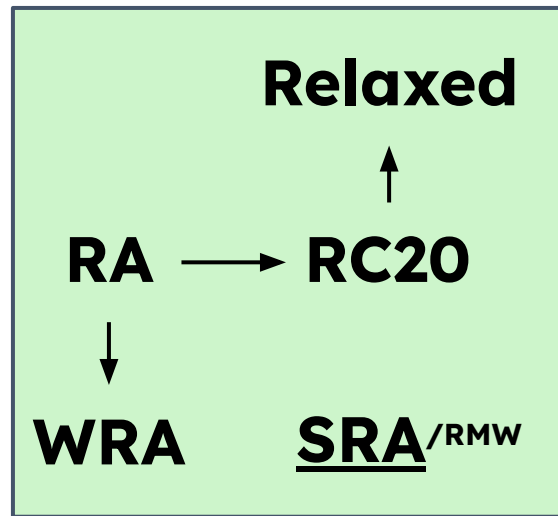


* unless Orthogonal Vectors can be solved in linear time

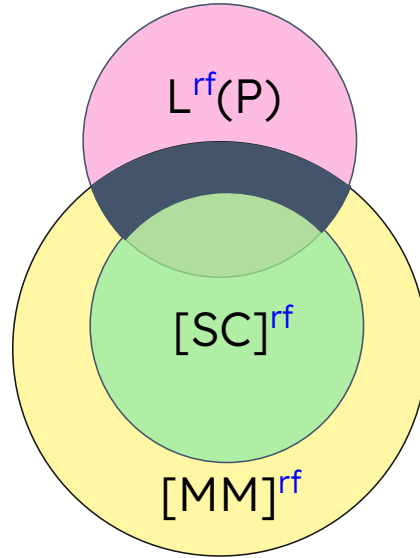
What are the
model-checking
implications?

Bounded treewidth verification

- May be **undecidable** in general
- Treewidth acts as an **under-approximation**
- Decidable for **every MSO-definable MM**



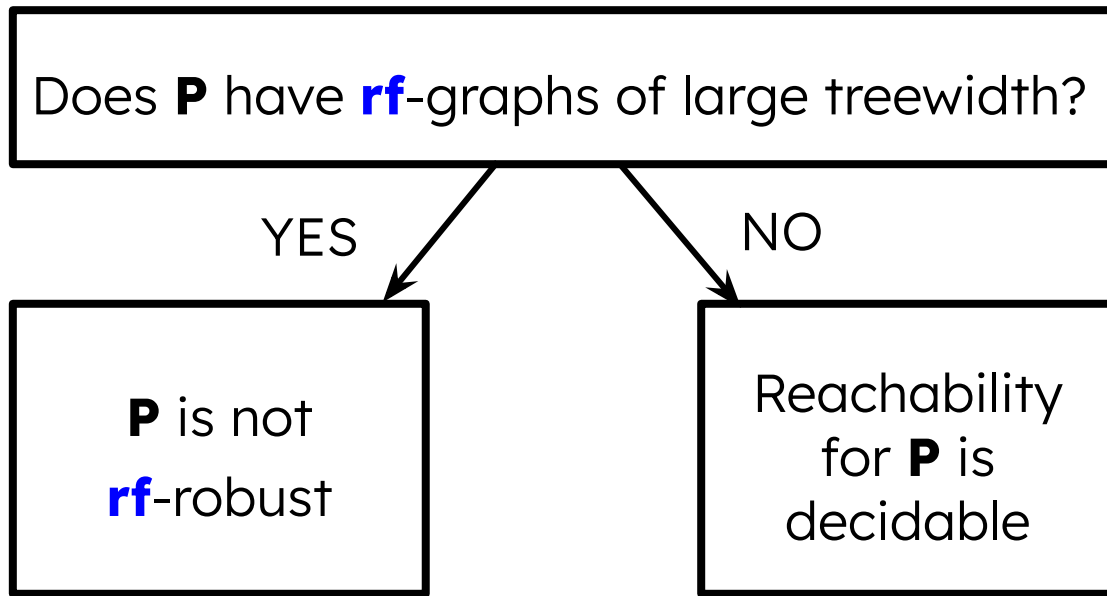
rf-robustness



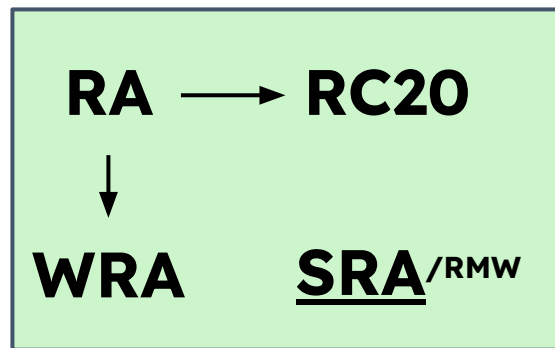
Is the shaded area empty?

Treewidth as a separator

Program **P** running under MSO-definable **MM**.



- We can devise an algorithm that solves **rf**-robustness **or** reachability
- Works for every MSO-definable **prefix-closed** MM
- **Works even if both problems are undecidable**



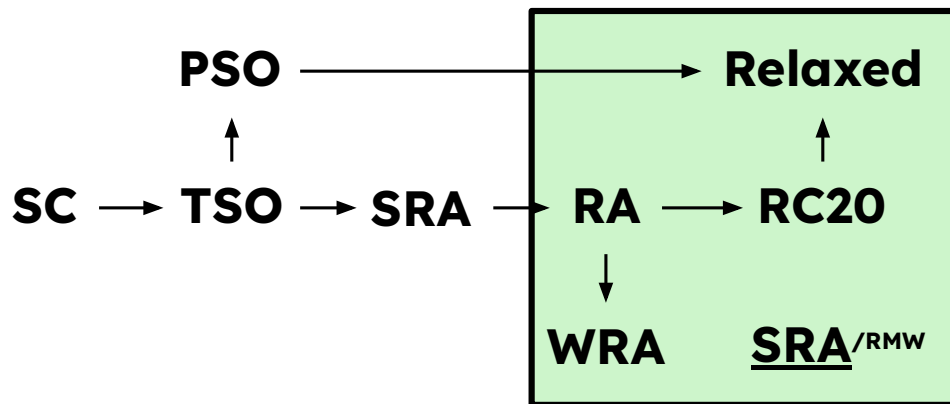
Takeaways

Treewidth

- SC has **rf**-graphs of bounded treewidth
- TSO and weaker models has **rf**-graphs of unbounded treewidth

MSO

- MSO-definability splits memory models into two classes.



Model-checking

- For MSO-definable memory models:
We show that reachability restricted to low-treewidth **rf**-graphs is decidable
- For prefix-closed MSO-definable memory models:
We devise an algorithm that, for a given program, solves either reachability **or rf**-robustness

Model-checking

- For MSO-definable memory models:
We show that reachability restricted to low-treewidth **rf**-graphs is decidable
- For prefix-closed MSO-definable memory models:
We devise an algorithm that, for a given program, solves either reachability **or rf**-robustness

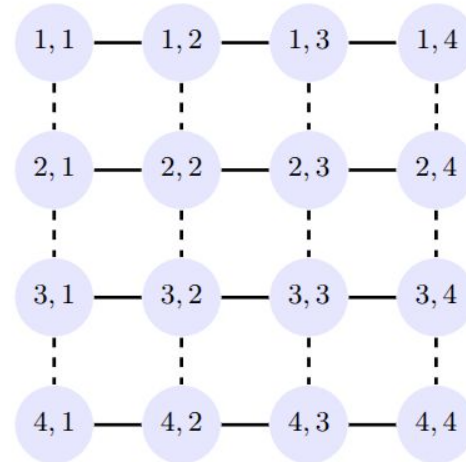
Thank you!

Appendix

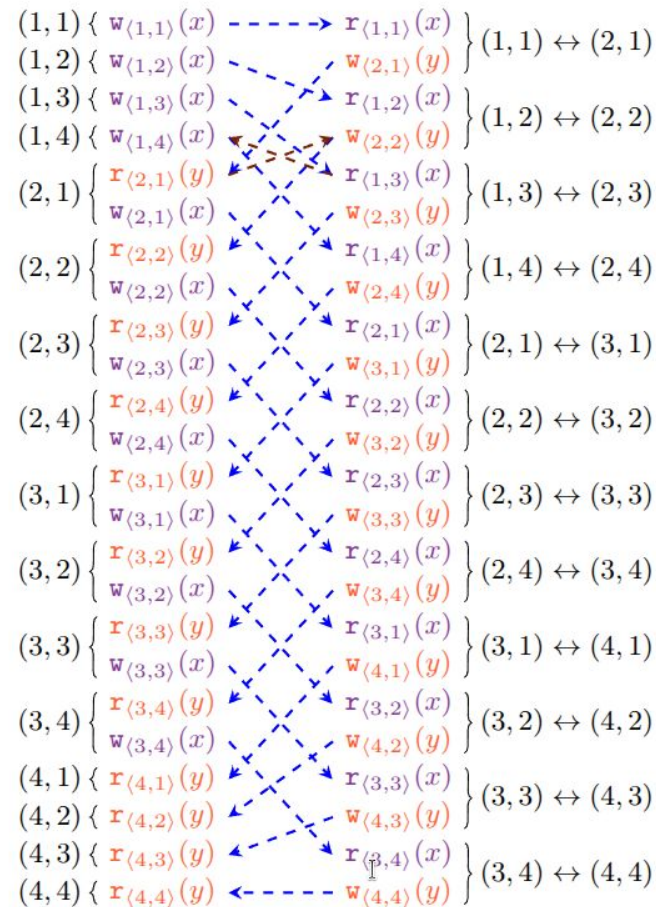
How do we show unbounded treewidth?

One way: show that the graph has a large grid minor

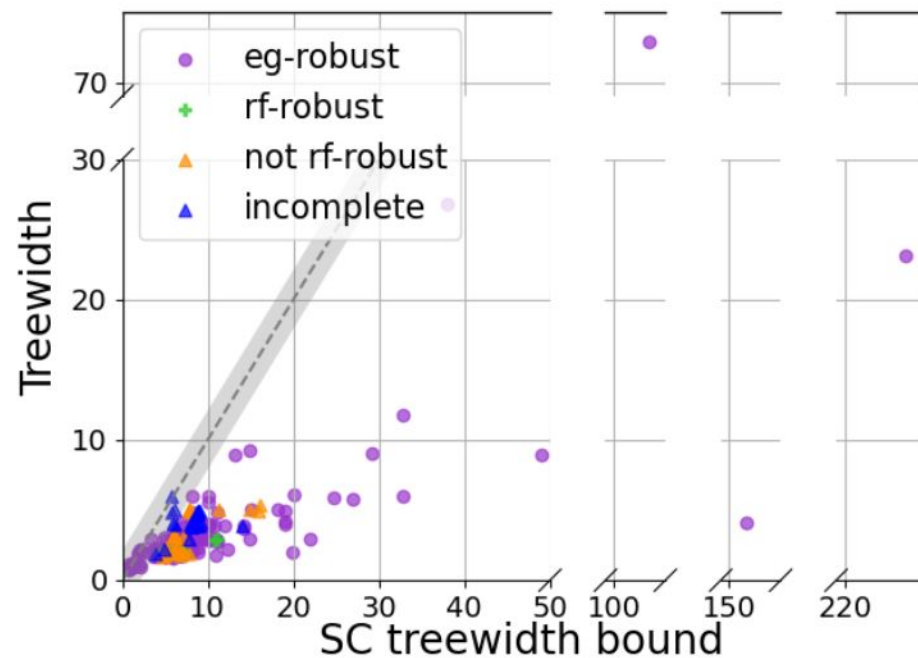
- Treewidth is closed under minors.
- Grids have high treewidth.



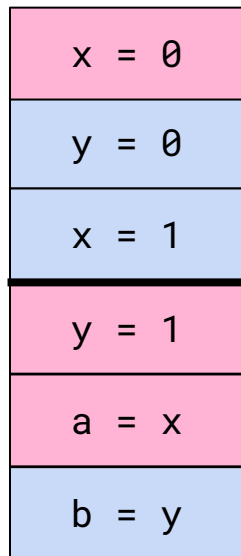
We construct a TSO-consistent
rf-graph G that has an
 arbitrarily large grid as a minor.



Treewidth is often low in practice



Why are SC **rf**-graphs simple?



Recall that an SC execution corresponds to simply interleaving the events of each thread in some order.

→ What do we need to remember at some program point?

- The latest event in each thread
- The latest write in each location

What does this mean for treewidth?

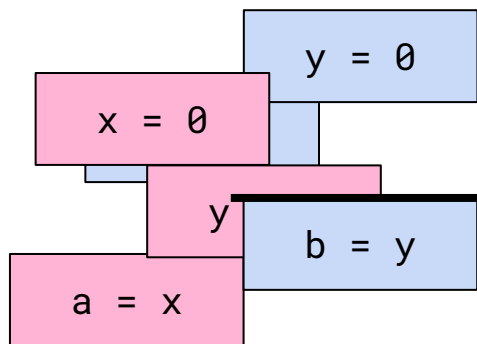
There is an order of events such that I only need to remember

- The latest event in each thread: **|Threads|** events
- The latest write in each location: **|Locations|** events

Yields:

The treewidth of any SC-consistent *rf*-graph is bounded by $|Threads| + |Locations|$

Why are weak **rf**-graphs less simple?



There is no simple order of events

What do we need to remember at some program point?

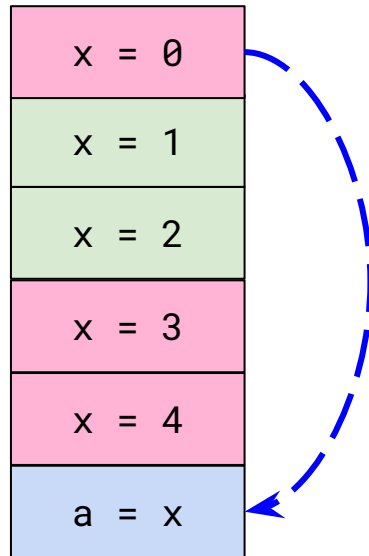
→ Possibly multiple (stale) writes

Can we still bound treewidth?

Treewidth and weak behaviours

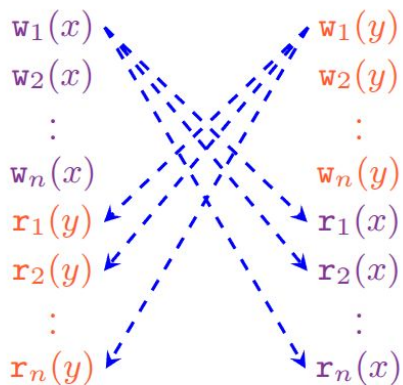
Even when $\mathbf{tw}(\mathbf{G}) > |\mathbf{Threads}| + |\mathbf{Locations}|$, treewidth bounds weak behaviours.

If $\mathbf{tw}(\mathbf{G}) < |\mathbf{Threads}| + k * |\mathbf{Locations}|$,
there is an ordering of events of $\mathbf{G}$
(following sb and rf), such that no event
reads a k -stale write.

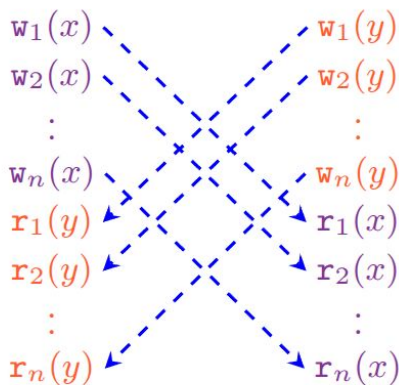


Treewidth and weak behaviours

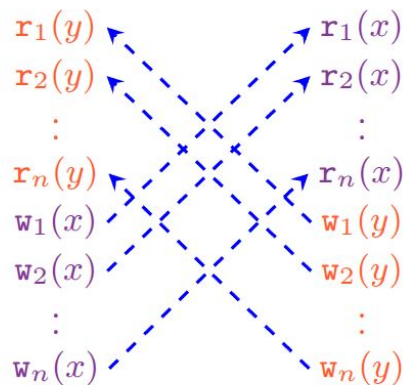
However, low treewidth does not mean exclusively strong behaviours



(a) Store Buffer



(b) Extended Store Buffer



(c) Extended Load Buffer

Why **rf**-robustness?

Ideal,
less tractable

State robustness: does this program reach the same program states under WMMs?

More sensitive,
more tractable

Execution-graph robustness: does this program produce the same execution graphs under WMMs?

Why **rf**-robustness?

Ideal,
less tractable

State robustness: does this program reach the same program states under WMMs?

rf-robustness: does this program produce the same rf-graphs under WMMs?

More sensitive,
more tractable

Execution-graph robustness: does this program produce the same execution graphs under WMMs?

